Python Logic Building

Zero to Advanced

Learn How to Think Like a Programmer

Problem → Understanding → Input → Output → Conditions → Steps → Algorithm → Pseudocode → Dry Run → Code

A structured, logic-first guide to problem solving in Python — for absolute beginners moving to advanced problem solving.

Table of Contents

Part 1 — Logic Kya Hai?

1.1 Programming Logic Kya Hota Hai?

Programming logic woh **step-by-step soch** hai jisse hum kisi real-world problem ko chhote-chhote instructions mein todte hain, jinhe computer follow kar sake. Code sirf uss soch ko likhne ka *tareeka* hai — asli kaam hai soch (logic) ko sahi banana.

Bahut se beginners yeh galti karte hain ki wo seedha code likhna shuru kar dete hain — bina yeh soche ki problem asal mein hai kya. Yeh aisa hi hai jaise koi bina map dekhe gaadi chalana shuru kar de. Result: wo beech mein confuse ho jaate hain, code baar-baar delete karte hain, aur frustrate ho jaate hain. Isliye is course ka pura focus hai — **pehle sochna, phir likhna**.

Ek simple tareeke se samjho: computer bilkul *bewakoof* hai. Wo sirf wahi karta hai jo usse bataya jaaye, aur bilkul exact wahi. Agar tumhari soch (logic) mein hi confusion hai, to computer bhi confuse result dega. Isliye "logic building" ka matlab hai — apni soch ko itna clear aur precise banana ki usmein koi bhi ambiguity na bache.

1.2 Logic aur Syntax Mein Farak

Yeh do cheezein bahut alag hain aur inko confuse karna beginners ki sabse badi problem hoti hai.

- **Logic** = Kya karna hai aur kis order mein karna hai (language-independent thinking). Logic tumhare dimaag mein hoti hai — isko tum Python, Java, ya sirf plain Hindi mein bhi explain kar sakte ho, kyunki yeh sirf *soch* hai, koi specific language nahi.
- **Syntax** = Us logic ko Python ke specific rules mein likhne ka tareeka — jaise colons (:) lagana, indentation sahi rakhna, keywords (if, for, while) ka sahi spelling. Syntax sirf ek "translation" hai — jaise ek hi khayal ko English mein ya Hindi mein likha ja sakta hai.
- Example: Ek insaan bina Python jaane bhi logic bata sakta hai: "Agar number 2 se pura divide ho jaaye (koi remainder na bache) to woh even hai." Yeh **logic** hai — koi bhi isse samajh sakta hai. Isko Python mein if n % 2 == 0: likhna **syntax** hai.
- Beginners jo galti karte hain: wo syntax yaad karne mein itna time lagate hain ki logic sochna bhool jaate hain. Lekin sach yeh hai — agar logic clear hai, syntax to Google/docs se 2 minute mein mil jaati hai. Isliye is course mein hum har baar pehle logic pe focus karenge, syntax uske baad.

1.3 Problem Ko Dekhkar Logic Kaise Sochein?

Jab bhi koi naya problem saamne aaye, iska matlab yeh nahi ki turant code likhna shuru kar do. Ek experienced programmer bhi pehle kuch der problem ko "samajhta" hai, sirf tab code likhta hai jab poori tarah clear ho jaaye ki karna kya hai.

- **Problem ko do baar dhyan se padho** — jaldi mat karo. Pehli baar general idea ke liye padho, dusri baar har word pe dhyan do — kayi baar ek chhota word ("at least", "exactly", "unique") poori problem ka meaning badal deta hai.
- **Jo bhi numbers/words diye hain unhe underline karo** — yeh aksar **input** hote hain. Jaise "ek list of 10 numbers di gayi hai" — yahan list aur 10 dono important clues hain.
- **Jo cheez pucha gaya hai use highlight karo** — yeh **output** hai. Aksar problem ke last sentence mein hota hai: "...batao", "...print karo", "...return karo".
- **Socho: kya koi condition hai** (agar X ho to Y, warna Z)? Words jaise "if", "agar", "jab", "unless" condition ka signal dete hain.

- **Socho: kya kaam repeat ho raha hai** (to loop lagega)? Words jaise "har", "each", "sabhi", "n baar" repetition ka signal dete hain.
- **Ek chhoti example khud, kaagaz par, haath se solve karke dekho** — yeh sabse underrated step hai. Agar tum khud haath se ek example solve nahi kar sakte, to code bhi nahi likh paoge, kyunki code bas tumhari manual soch ko automate karta hai.

1.4 Real-Life Examples Se Samjho

Socho tum chai bana rahe ho. Tumhara "algorithm" kuch aisa hoga: paani garam karo → chai patti daalo → doodh daalo → chini daalo → 2 minute ubalo → chhaan lo. Yeh exact wahi cheez hai jo programming mein hoti hai — ek problem ("chai chahiye") ko clear, ordered steps mein todna. Programming mein bas ingredients ki jagah **data** hota hai aur process ki jagah **code**.

Ek aur example: Google Maps. Jab tum kisi jagah jaane ka route poochte ho, Maps kya karta hai? (1) Tumhara current location (**input**) leta hai, (2) Destination (**input**) leta hai, (3) Possible routes compare karta hai (**conditions/logic** — konsa fastest hai, traffic kaisa hai), (4) Best route batata hai (**output**). Yeh bilkul wahi 4 cheezein hain jo har programming problem mein hoti hain: Input → Processing/Logic → Output.

Teesra example — ATM machine. Jab tum paise nikaalte ho: Input = amount jo nikaalna hai. Conditions = kya balance kaafi hai? Kya amount 100 ke multiple mein hai? Processing = balance se amount minus karna. Output = cash dena + naya balance dikhana. Agar tum kabhi confuse ho ki "is problem mein logic kaise socho", to yeh 4-step frame (Input → Conditions → Processing → Output) hamesha kaam aayega.

1.5 Universal Format — Har Problem Isi Tarah Solve Hoga

Step	Matlab
Problem	Kya poocha gaya hai (raw statement)
Understanding	Apne shabdon mein dobara likhna
Input	Kya diya gaya hai
Output	Kya return/print karna hai
Conditions	Kaunse rules/cases handle karne hain
Steps	Kaam ko chhote actions mein todna
Algorithm	Steps ka ordered, logical sequence
Pseudocode	English/Hinglish mein code jaisa structure
Dry Run	Haath se ek example chalake verify karna
Code	Ab Python mein likhna

1.6 The 14-Step Problem Solving Process

Har problem — chahe wo beginner ho ya advanced — is process se guzarni chahiye. Shuru mein yeh slow lagega, lekin yehi habit tumhe expert banayegi.

1. Read carefully — Problem 2 baar padho. Pehli baar overview ke liye, dusri baar detail ke liye.
2. Apni language mein samjho — Problem ko khud ko explain karo jaise kisi dost ko bata rahe ho.
3. Input identify karo — Kya data milega? Type kya hoga — number, string, list?
4. Output identify karo — Exact result kya chahiye? Konsa format?
5. Conditions identify karo — Kaunse special cases/rules hain?
6. Problem todo — Bade kaam ko 2-4 chhote sub-kaam mein baato.
7. Variables identify karo — Kaunsi values store karni padengi?
8. Algorithm banao — Step-by-step plan — English mein.
9. Pseudocode likho — Code jaisa structure, bina syntax ki tension ke.
10. Dry run karo — Kaagaz par ek example se apna algorithm test karo.
11. Python code likho — Ab pseudocode ko Python mein convert karo.
12. Code test karo — Multiple normal inputs se check karo.
13. Edge cases check karo — Empty input, negative number, 0, duplicate — yeh sab try karo.
14. Optimize karo — Kya isse simpler/faster likha ja sakta hai?

Example: "Largest of Two Numbers" Is Process Se

Problem: Do numbers diye hain, bada number batao.
Understanding: Do numbers compare karke jo bada hai wo return karna hai.
Input: a, b (dono integers).
Output: Bada number.
Conditions: Agar dono barabar hain to?
Algorithm: a aur b compare karo → jo bada ho use return karo → agar equal ho to bata do equal hain.

LEVEL 1

Beginner Logic Building

Variables, Conditions, Comparison & Decision-Making Problems

A. Variables & Basic Thinking

Variable ko samajhne ka sabse aasan tareeka hai — ise ek **labelled box** ki tarah socho jismein tum koi value rakh sakte ho, aur baad mein us box ka naam lekar wahi value wapas nikaal sakte ho. Box ka content change ho sakta hai (isliye "variable" naam hai) lekin box ka naam wahi rehta hai.

- **Variable identify karna:** Problem mein jo bhi "changing value" hai jise tumhe baad mein use karna hai — wahi variable banta hai. E.g. "student ke 3 marks add karo" → marks1, marks2, marks3, total — sab variables hain. Trick yeh hai: agar tum us value ko kisi naam se bula sakte ho (jaise "total", "age", "price"), to woh variable banegi.
- **Kya store karein:** Agar value ek hi baar use honi hai, variable ki zarurat nahi. Agar value baar-baar chahiye ya update hone wali hai, use variable mein rakho. Jaise ek loop mein "sum" baar-baar update hota hai — isliye woh variable hai. Lekin agar tum sirf ek baar print(5 + 3) karna chahte ho, wahan variable ki zarurat nahi.
- **Input/Output thinking:** Hamesha socho — "Yeh value user degi (input) ya main calculate karunga (derived)?" Input variables user se aati hain (jaise input() se), derived variables calculation se banti hain (jaise area = length * width).
- **Naming matters:** Variable ka naam meaningful hona chahiye. x ki jagah total_marks likhna — kyun? Kyunki jab tumhara code 50 lines ka ho jaayega, tumhe khud yaad nahi rahega x kya represent karta tha. Ek achha naam khud documentation ka kaam karta hai.

```
# Simple calculation example
length = 5
width = 3
area = length * width   # derived variable
print(area)
```

Yahan length aur width "input-like" values hain (diye gaye hain), aur area ek **derived** variable hai jo calculation se banti hai. Isi soch ko har problem mein apply karo: pehle socho konsi values diye gaye hain, phir socho konsi values tumhe khud calculate karni padengi.

B. Conditions — if / elif / else

Conditions programming mein "decision making" ka core hain. Real life mein bhi hum hamesha conditions ke through decisions lete hain: "Agar baarish ho rahi hai, to chhata le lo, warna nahi." Python mein yehi soch if...else ban jaati hai.

- **if**: Sirf tab chalta hai jab condition True ho. Jaise "agar marks 40 se zyada hain" — yeh sirf tabhi execute hoga jab condition sahi ho.
- **elif** (else-if ka short form): Pehli condition False ho to agla check karta hai. Ek if block mein multiple elif ho sakte hain — Python unhe top se bottom order mein check karta hai aur jo pehli True milti hai, sirf uska block chalta hai, baaki skip ho jaate hain.

- **else**: Sab conditions (if aur saare elif) False hone par yeh chalta hai — yeh ek "default" case hai. else mein koi condition nahi likhi jaati.
- **Nested if**: Ek if ke andar dusra if — jab do levels ka decision lena ho. Jaise pehle check karo "kya age 18+ hai", agar haan to andar check karo "kya ID card hai".
- **AND**: Dono (ya saari) conditions True honi chahiye tabhi poora expression True banta hai. Agar ek bhi False hui, poora False.
- **OR**: Conditions mein se koi bhi ek True ho to poora expression True ban jaata hai. Sirf tab False hota hai jab saari conditions False hon.
- **NOT**: Condition ki value ko ulta kar deta hai — True ko False, aur False ko True.

Ek chhota truth-table jaisa example: agar age=20 aur has_id=True, to age >= 18 and has_id → True and True → **True**. Agar has_id=False hota, to True and False → **False** — matlab poora condition fail ho jaata, chahe age sahi ho.

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Voting allowed")
else:
    print("Not allowed")
```

■ **Common Mistake:** Beginners aksar = (assignment) aur == (comparison) mein confuse ho jaate hain. x = 5 ka matlab hai "x mein 5 rakho". x == 5 ka matlab hai "kya x, 5 ke barabar hai?". if condition mein hamesha == use hota hai.

C. Comparison & Decision-Making Problems

In sab problems mein pehle khud sochne ki koshish karo — pattern ek jaisa hai: condition socho → order decide karo → code likho.

Problem 1: Number Positive / Negative / Zero

Thinking (khud se poochho):

- Number kis type ka ho sakta hai — int ya float dono?
- 0 ko positive maanoge ya alag case?

Logic:

- Agar n > 0 → positive
- Agar n < 0 → negative
- Agar n == 0 → zero

Pseudocode:

```
INPUT n
IF n > 0:
    PRINT "Positive"
ELIF n < 0:
    PRINT "Negative"
ELSE:
    PRINT "Zero"
```

Dry Run:

n	Condition Checked	Result
-5	n > 0 ? No → n < 0 ? Yes	Negative
0	n > 0 ? No → n < 0 ? No	Zero

Python Code:

```
n = int(input("Enter number: "))
if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")
```

Explanation:

Order important hai — pehle > 0 check karo, phir < 0, warna last mein automatically 0 bachta hai.

■ **Why this works:** elif ka use ek hi baar mein exactly ek branch chalata hai — agar hum teen alag if statements (bina elif) likhte, to Python har ek ko independently check karta, jo unnecessary aur kabhi-kabhi buggy ho sakta hai jab conditions overlap karti hain.

> ■ **Common Mistake:** Agar tum n > 0 aur n < 0 dono ko separate if statements (elif ki jagah) mein likh do, to bhi output sahi aayega is case mein, lekin yeh galat practice hai — jaise-jaise conditions badhengi, bugs aane ka chance badh jaata hai.

Practice:

- Check karo number float ho to bhi kaam karta hai kya?
- Bina elif ke, sirf nested if se likho.

Problem 2: Even or Odd

Thinking (khud se poochho):

- Modulo operator (%) kya karta hai?
- Negative number even/odd ho sakta hai kya?

Logic:

- Agar n % 2 == 0 → Even, warna Odd

Pseudocode:

```
INPUT n
IF n % 2 == 0:
    PRINT "Even"
ELSE:
    PRINT "Odd"
```

Dry Run:

n	n % 2	Result
7	1	Odd
10	0	Even

Python Code:

```
n = int(input())
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Explanation:

% operator division ke baad ka remainder deta hai — remainder 0 ka matlab hamesha even.

Practice:

- User se 5 numbers lo aur ek saath sabke even/odd batao.

Problem 3: Largest of Three Numbers

Thinking (khud se poochho):

- Kya sab numbers equal ho sakte hain?
- Do conditions mein compare karne ka better tareeka kya hai?

Logic:

- a ko sabse pehle b se, phir winner ko c se compare karo
- Ya Python ka built-in max() bhi use ho sakta hai (optimized version)

Pseudocode:

```
IF a >= b AND a >= c:
    largest = a
ELIF b >= a AND b >= c:
    largest = b
ELSE:
    largest = c
```

Dry Run:

a	b	c	a>=b & a>=c?	Result
4	9	2	No	largest = b = 9

Python Code:

```
a, b, c = 4, 9, 2
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print(largest)

# Optimized version:
print(max(a, b, c))
```

Explanation:

Pehle brute-force (manual comparison) samjho, phir dekho built-in max() wahi kaam kaise 1 line mein karta hai — yeh "optimize karna" ka pehla example hai.

Practice:

- 4 numbers ka largest nikalo bina max() use kiye.
- Smallest of three numbers nikalo.

Problem 4: Leap Year Check

Thinking (khud se poochho):

- Leap year ka rule kya hai — sirf 4 se divide hona kaafi hai?
- Century years (jaise 1900, 2000) ka special rule kya hai?

Logic:

- Year 4 se divisible ho AND (100 se divisible na ho OR 400 se divisible ho)

Pseudocode:

```
IF year % 4 == 0 AND (year % 100 != 0 OR year % 400 == 0):
    PRINT "Leap Year"
ELSE:
    PRINT "Not Leap Year"
```

Dry Run:

year	%4==0	%100!=0 or %400==0	Result
1900	True	False	Not Leap
2000	True	True	Leap

Python Code:

```
year = int(input())
if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
    print("Leap Year")
else:
    print("Not Leap Year")
```

Explanation:

Yeh problem sikhati hai ki real-world rules mein multiple conditions AND/OR se combine hoti hain — is case mein ek exception ke andar bhi ek exception hai.

Practice:

- 1600, 1800, 2024, 2100 test karo aur pehle predict karo answer, phir code chalao.

Level 1 — Practice Set (bina hint ke try karo)

- Pass/Fail: marks >= 40 to Pass warna Fail.
- Grade Calculator: 90+ A, 75-89 B, 60-74 C, warna D.
- Simple Calculator: user se operator (+,-,*,/) le kar calculation karo.
- Login Validation: username aur password dono match karne par "Login Successful".
- Voting Eligibility: age aur citizenship dono check karo.

LEVEL 2

Loops & Pattern Logic

while, for, range(), break, continue, nested loops, patterns

Loop Thinking — Sirf Syntax Nahi

Loops ka core idea bahut simple hai: **koi bhi kaam jo baar-baar repeat ho raha hai, usse haath se n baar likhne ke bajaye, ek hi baar likh kar computer ko keh do ki ise n baar repeat karo.** Jaise agar tumhe 1 se 100 tak print karna hai, to 100 print statements likhna galat approach hai — ek loop se 3 lines mein ho jaata hai.

- **Kab loop use karna hai?** Jab ek kaam multiple baar repeat ho raha ho — chahe fixed baar ("5 baar karo") ya kisi condition tak ("jab tak user 'quit' na type kare").
- **Kitni baar chalega?** Ya to fixed count pata hoga (for loop) ya condition pe depend karega (while loop). Yeh decide karna sabse pehla step hai — agar count pata hai, for loop use karo; agar pata nahi kitni baar chalega (depends on user input ya kisi event pe), while loop use karo.
- **Starting value:** Loop kis number/state se shuru hoga — usually 0, 1, ya list ka pehla element. Yeh problem pe depend karta hai — agar 1 se count karna hai (jaise "1st student"), start 1 se hoga; agar list ka index access karna hai, start 0 se hoga (Python 0-indexed hai).
- **Ending condition:** Loop kab rukega — range ki limit ya condition False hona. Yeh socho sabse zyada dhyan se, kyunki galat ending condition se ya to loop ek baar kam/zyada chalta hai (off-by-one error), ya kabhi rukta hi nahi (infinite loop).
- **Counter:** Woh variable jo track karta hai loop kitni baar chala (i, count). Yeh usually har iteration mein 1 se badhta hai.
- **Accumulator:** Woh variable jo values ko collect/add karta hai (total, sum, result). Iski starting value bahut important hai — sum ke liye 0 se start karo, multiplication ke liye 1 se (kyunki kisi bhi number ko 0 se multiply karo to 0 hi milega).

■ **Common Mistake:** Sabse common loop mistake: accumulator ko loop ke *andar* reset kar dena. Agar total = 0 loop ke andar likh doge, to har iteration mein total wapas 0 ho jaayega aur sirf last value store hogi. Accumulator ko hamesha loop shuru hone se *pehle*, sirf ek baar initialize karo.

```
# Counter vs Accumulator
count = 0      # counter - kitni baar loop chala
total = 0      # accumulator - values jama ho rahi hain

for i in range(1, 6):
    count += 1
    total += i

print(count, total)  # 5 15
```

Number Problems

Problem: Factorial of a Number

Thinking (khud se poochho):

- Factorial 0 aur 1 ka kya hota hai?
- Negative number ka factorial possible hai kya?

Logic:

- fact = 1 se start karo (multiplication ka identity)
- 1 se n tak har number ko multiply karte jao

Pseudocode:

```
fact = 1
FOR i FROM 1 TO n:
    fact = fact * i
PRINT fact
```

Dry Run:

i	fact before	calculation	fact after
1	1	1*1	1
2	1	1*2	2
3	2	2*3	6
4	6	6*4	24

Python Code:

```
n = 4
fact = 1
for i in range(1, n + 1):
    fact *= i
print(fact)
```

Explanation:

fact ek accumulator hai jo multiplication collect karta hai — sum ke accumulator (total=0) se alag hai kyunki yahan starting value 1 honi chahiye, 0 nahi (0 se multiply karne pe sab 0 ho jayega).

Practice:

- Factorial ko while loop se bhi likho.
- 0! aur 1! test karo — kya answer 1 aata hai?

Problem: Check Prime Number

Thinking (khud se poochho):

- 1 prime hai kya?
- 2 se check start karein ya 1 se?
- Kya sirf n/2 tak check karna kaafi hai (optimization)?

Logic:

- n < 2 ho to prime nahi
- 2 se n-1 tak koi bhi number n ko fully divide kare to prime nahi
- Koi divisor na mile to prime hai

Pseudocode:

```
IF n < 2: NOT PRIME
is_prime = True
FOR i FROM 2 TO n-1:
    IF n % i == 0:
        is_prime = False
        BREAK
PRINT is_prime
```

Dry Run:

i	n % i	is_prime
2	1 (n=7)	True
3	1	True
4..6	!=0	True

Python Code:

```
n = 7
is_prime = True
if n < 2:
    is_prime = False
else:
    for i in range(2, n):
        if n % i == 0:
            is_prime = False
            break
print(is_prime)

# Optimized: sirf sqrt(n) tak check karna kaafi hai
import math
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True
```

Explanation:

break yahan important hai — ek divisor milte hi loop rokna chahiye, waste iterations avoid karne ke liye. sqrt(n) tak check karna badi optimization hai bade numbers ke liye.

Practice:

- is_prime(2), is_prime(1), is_prime(97) test karo.
- 1 se 50 tak sab prime numbers print karo.

Problem: Palindrome Number

Thinking (khud se poochho):

- Number ko reverse kaise karein loop se?
- Negative number palindrome ho sakta hai kya?

Logic:

- Number ka reverse banao (digit by digit)
- Reverse ko original se compare karo

Pseudocode:

```
original = n
reversed_num = 0
WHILE n > 0:
    digit = n % 10
    reversed_num = reversed_num * 10 + digit
    n = n // 10
IF reversed_num == original: PALINDROME
```

Dry Run:

n	digit	reversed_num
121	1	1
12	2	12
1	1	121

Python Code:

```
n = 121
original = n
reversed_num = 0
while n > 0:
    digit = n % 10
    reversed_num = reversed_num * 10 + digit
    n //= 10
print(reversed_num == original)
```

Explanation:

n % 10 se last digit milta hai, n // 10 se last digit hat jaata hai. reversed_num ko multiply-then-add se hum digits ko ulti order mein jodte hain.

Practice:

- Sum of digits nikalo isi loop pattern se.
- Count digits nikalo (kitni baar loop chala).
- Armstrong number check karo (digits ke cubes ka sum == number).

Pattern Problems — Rows aur Columns Ki Soch

- **Rows kaise identify karein:** Outer loop = kitni rows print honi hain (usually 1 to n).
- **Columns kaise identify karein:** Inner loop = har row mein kitne characters print honge — yeh row number pe depend kar sakta hai.
- **Outer loop kya karega:** Row ko control karega aur har row ke baad newline karega.
- **Inner loop kya karega:** Us row ke andar characters print karega.
- **Manually observe karo:** Pattern ko haath se 3-4 rows likho aur dekho row number aur stars/spaces ke beech kya relation hai.

Pattern: Right Triangle of Stars (n=4)

Thinking (khud se poochho):

- Row 1 mein kitne stars? Row 2 mein? Pattern kya hai?

Logic:

- Row number = us row mein stars ki ginti

Pseudocode:

```
FOR i FROM 1 TO n:
    FOR j FROM 1 TO i:
        PRINT "*"
    NEWLINE
```

Dry Run:

Outer i	Inner j runs	Row Output
1	1	*
2	1,2	**
3	1,2,3	***
4	1,2,3,4	****

Python Code:

```
n = 4
for i in range(1, n + 1):
    for j in range(i):
        print("*", end="")
    print()
```

Explanation:

Outer loop row control karta hai (i = 1 to n), inner loop (j = 0 to i-1, matlab i baar) stars print karta hai. Yeh sabse basic pattern hai — isse hi pyramid, number patterns nikalte hain.

Practice:

- Isi pattern ko numbers se print karo (1, 12, 123...).
- Reverse triangle banao (n stars se 1 star tak).

- Pyramid pattern try karo (spaces + stars combine karke).

Level 2 — Practice Set

- Multiplication table of n.
- Sum of first n natural numbers.
- Fibonacci series up to n terms.
- Perfect number check (divisors ka sum == number).
- Number pyramid pattern.
- Reverse number aur usse original se compare karna.

LEVEL 3

Intermediate Logic

Strings, Lists, Tuples, Sets, Dictionaries — Real Problem-Solving

Yahan Se Pehle Khud Sochna Hai

Level 1-2 mein problems ke inputs simple the (ek number, kuch numbers). Ab hum **collections of data** (lists, strings, dictionaries) ke saath kaam karenge, jahan edge cases zyada important ho jaate hain — jaise "list empty hui to?", "duplicate values aayi to?". Is level se main tumse pehle clarifying questions poochunga, jaise ek senior developer poochta hai requirements clear karne ke liye. Tab tak code nahi likhna jab tak edge cases clear na ho jayein — yeh ek professional habit hai jo tumhe real projects mein bahut kaam aayegi.

Data structures (list, tuple, set, dictionary) ko ek tareeke se socho — yeh sab "boxes" hain jinme multiple values rakhi ja sakti hain, lekin har box ki apni properties hain: list mein order matter karta hai aur duplicates allowed hain; set mein sirf unique values rehti hain aur order guaranteed nahi hota; dictionary mein har value ek "key" (naam) ke saath judi hoti hai. Level 7 mein hum inko detail mein compare karenge.

Problem: Second Largest Number in a List

Thinking (khud se poochho):

- Kya list empty ho sakti hai? (Agar haan → handle karna padega)
- Duplicate values allowed hain kya? (e.g. [5,5,3] mein second largest kya hai?)
- Kya sirf ek hi distinct value ho sakti hai puri list mein?
- "Second largest" ka matlab 2nd position ka element hai ya 2nd distinct largest value?

Logic:

- Pehle list ko duplicates hata kar unique values mein convert karo (agar distinct chahiye)
- Sort karke ya do variables (largest, second_largest) track karke nikalo

Pseudocode:

```
REMOVE duplicates -> unique_list
SORT unique_list DESCENDING
RETURN unique_list[1]
```

Dry Run:

List	Unique Sorted Desc	Second Largest
[5,5,3,8,8,2]	[8,5,3,2]	5

Python Code:

```
nums = [5, 5, 3, 8, 8, 2]
unique = list(set(nums))
unique.sort(reverse=True)
if len(unique) < 2:
    print("No second largest")
else:
    print(unique[1])

# Optimized: ek hi pass mein (bina sort/set ke), O(n)
def second_largest(nums):
    first = second = float('-inf')
    for n in nums:
        if n > first:
            second = first
            first = n
        elif first > n > second:
            second = n
    return second
```

Explanation:

set() se duplicates hatte hain, phir sort karke second element milta hai — lekin yeh O(n log n) hai. Optimized version single pass mein O(n) time mein do variables track karke karta hai.

Practice:

- Kya code negative numbers ke liye kaam karega?
- Agar list mein sirf ek hi unique value ho to kya output aana chahiye?

Problem: Check Anagram (Do Strings)

Thinking (khud se poochho):

- Case-sensitive check karna hai ya nahi? ("Listen" vs "listen")
- Spaces ko ignore karna hai kya?
- Same letters, same count — yehi anagram ki definition hai, confirm karo.

Logic:

- Dono strings ko lowercase karo aur sort karo — agar equal hain to anagram hai

Pseudocode:

```
s1 = sort(lower(s1))
s2 = sort(lower(s2))
RETURN s1 == s2
```

Dry Run:

s1	s2	sorted s1	sorted s2	Anagram?
listen	silent	eilnst	eilnst	True

Python Code:

```
s1, s2 = "listen", "silent"
is_anagram = sorted(s1.lower()) == sorted(s2.lower())
print(is_anagram)

# Optimized using frequency counting (Counter) - O(n) instead of O(n log n)
from collections import Counter
is_anagram = Counter(s1.lower()) == Counter(s2.lower())
```

Explanation:

Sorting approach simple hai lekin O(n log n) hai. Counter (hashing/frequency-count) approach O(n) hai — yahan se "hashing" ka concept intro hota hai jo Level 6 mein detail mein aayega.

Practice:

- Spaces wale strings ke liye test karo: "Dormitory" vs "Dirty Room".
- Frequency dictionary khud (bina Counter ke) bana kar solve karo.

Level 3 — Practice Set

- Character frequency counter in a string.
- Remove duplicates from a list.
- Reverse a string without using [::-1].
- Find common elements between two lists.
- Find the missing number in a list of 1 to n.
- Find all duplicate elements in a list.

LEVEL 4

Functions & Modular Thinking

Breaking large problems into small, reusable pieces

- **Function kab banao:** Jab koi kaam bar-bar repeat ho, ya jab ek bade problem ka ek clear, standalone sub-task ho.
- **Parameters vs Arguments:** Parameter function define karte waqt ka naam hai; argument woh actual value hai jo call karte waqt di jaati hai.
- **Return:** Function se value wapas bhejna — print() sirf dikhata hai, return use karne layak deta hai.
- **Function composition:** Ek function ka output dusre function ka input bann sakta hai.

Example Project: Expense Tracker (Architecture First)

Problem: User apne expenses add kar sake, unhe dekh sake, total nikaal sake, aur category-wise summary dekh sake. Pehle sochte hain **architecture** — code likhne se pehle.

- **add_expense(expenses, amount, category)** — naya expense list mein add karta hai.
- **view_expenses(expenses)** — sab expenses print karta hai.
- **calculate_total(expenses)** — sab amounts ka sum return karta hai.
- **category_summary(expenses)** — category-wise total dictionary return karta hai.
- **delete_expense(expenses, index)** — ek specific expense remove karta hai.

Data Structure Choice: Har expense ek dictionary hoga {'amount': 200, 'category': 'Food'}, aur sab expenses ek list mein rahenge — yeh list of dictionaries pattern bahut common hai real projects mein.

```
expenses = []

def add_expense(expenses, amount, category):
    expenses.append({"amount": amount, "category": category})

def calculate_total(expenses):
    total = 0
    for e in expenses:
        total += e["amount"]
    return total

def category_summary(expenses):
    summary = {}
    for e in expenses:
        cat = e["category"]
        summary[cat] = summary.get(cat, 0) + e["amount"]
    return summary

add_expense(expenses, 200, "Food")
add_expense(expenses, 500, "Travel")
print(calculate_total(expenses))       # 700
print(category_summary(expenses))      # {'Food': 200, 'Travel': 500}
```

Yeh dikhata hai **modular thinking** — har function ek chhota, clearly-named kaam karta hai. Isse code padhna aur test karna dono aasan ho jaate hain.

LEVEL 5

Intermediate Problem-Solving Projects

Real-world style mini-projects using the full logic framework

Har project ke liye yehi format follow hoga: **Requirements** → **Breakdown** → **Data** → **Functions** → **Algorithm** → **Pseudocode** → **Code** → **Testing** → **Edge Cases** → **Improvements.** Neeche har project ka "starter blueprint" hai — chat mein hum inhe ek-ek karke poora banayenge.

ATM Simulation — Balance check, withdraw, deposit karna

Data: balance (number), pin (number). Functions: check_balance(), withdraw(amount), deposit(amount), validate_pin(). Edge case: negative amount, insufficient balance.

Number Guessing Game — Computer ek random number soche, user guess kare

Data: secret_number, attempts. Functions: generate_number(), check_guess(). Edge case: invalid (non-numeric) input.

Contact Book — Naam-number save, search, delete karna

Data: dictionary {name: number}. Functions: add_contact(), search_contact(), delete_contact(). Edge case: duplicate naam, contact na milna.

Student Management System — Students ke marks store aur analyze karna

Data: list of dicts {name, marks}. Functions: add_student(), average_marks(), topper(). Edge case: empty student list.

Bank Account Simulation — Multiple accounts, transactions

Data: dictionary {account_no: balance}. Functions: create_account(), transfer(), transaction_history(). Edge case: transfer to non-existent account.

Inventory Management — Products, stock quantity track karna

Data: dict {product: {price, quantity}}. Functions: add_stock(), sell_item(), low_stock_alert(). Edge case: selling more than available stock.

To-Do List — Tasks add, complete, delete karna

Data: list of dicts {task, done}. Functions: add_task(), mark_done(), remove_task(). Edge case: marking a non-existent task done.

Password Validator — Strong password rules check karna

Data: password string. Functions: has_uppercase(), has_digit(), has_special_char(), is_strong(). Edge case: empty password.

LEVEL 6

Advanced Logic Building

Complexity, Recursion, Two Pointers, Sliding Window, Hashing, DP

Time & Space Complexity (Big O) — Simplified

Ab tak humne "logic sahi hai ya nahi" pe focus kiya. Lekin ek advanced programmer sirf yeh nahi sochta ki solution kaam karta hai — wo yeh bhi sochta hai ki **solution kitna fast hai jab data bahut bada ho jaaye**. Yehi "complexity" ka concept hai.

Ek real-life analogy: socho tumhe ek phone book mein ek naam dhundna hai. Agar tum page 1 se shuru karke ek-ek naam check karte jao ("linear search"), 1000 logon ki book mein worst case 1000 checks lagenge. Lekin agar phone book alphabetically sorted hai, tum beech se shuru karke har baar aadha hissa chhod sakte ho ("binary search") — 1000 logon mein sirf ~10 checks lagenge! Yehi farak Big O measure karta hai — kitna kaam badhta hai jab data badhta hai.

Big O batata hai ki input badhne par tumhara code kitna slow hota hai. Isse hum "kaunsa approach better hai" scientifically decide kar sakte hain, sirf guess nahi.

Notation	Matlab (Simple)	Example
O(1)	Input size se farak nahi padta	list[0] access karna
O(log n)	Har step mein aadha data chhod dena	Binary Search
O(n)	Ek baar poora data dekhna	Simple loop se sum nikalna
O(n log n)	Har element ke liye log n kaam	Efficient sorting (merge sort)
O(n^2)	Har element ke liye poore data ko dekhna	Nested loop se duplicates dhundna

Recursion — Beginner Example

Recursion tab use hota hai jab ek problem apne hi chhote version mein todi ja sake. Har recursive function ko do cheezein chahiye: **base case** (kab rukna hai) aur **recursive case** (chhote problem par khud ko call karna).

```
def factorial(n):
    if n <= 1:            # base case
        return 1
    return n * factorial(n - 1)   # recursive case

print(factorial(5))   # 120
```

Two Pointers — Beginner Example

Jab hume ek sorted list mein do elements dhundne hon jinka sum ek target ho, to dono sirre se ek-ek pointer chalate hain — brute force O(n^2) se O(n) tak aa jaate hain.

```
def has_pair_with_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return True
        elif s < target:
            left += 1
        else:
            right -= 1
    return False

print(has_pair_with_sum([1,2,3,4,6], 10))  # True (4+6)
```

Sliding Window — Beginner Example

Jab hume ek fixed-size ya variable-size "window" (continuous part) ka sum/average chahiye ho, to har baar poora recompute karne ke bajaye sirf window ke edges update karte hain.

```
def max_sum_subarray(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]   # naya element add, purana hataa
        max_sum = max(max_sum, window_sum)
    return max_sum
```

Hashing — Beginner Example

Dictionary/set use karke O(1) mein check karna ki koi value pehle dekhi hai ya nahi — isse nested loops (O(n^2)) avoid karte hain.

```
def has_duplicate(arr):
    seen = set()
    for num in arr:
        if num in seen:
            return True
        seen.add(num)
    return False
```

Basic Dynamic Programming — Fibonacci Example

Plain recursion mein Fibonacci same values baar-baar calculate karta hai — DP unhe "yaad" (memoize) rakhta hai taaki dobara calculate na karna pade.

```
def fibonacci(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci(n-1, memo) + fibonacci(n-2, memo)
    return memo[n]
```

LEVEL 7

Data Structures & Algorithmic Thinking

Choosing the right tool for the job

Data Structure	Kab Use Karein
List	Order matter karta hai, duplicates allowed, index se access chahiye
Tuple	Data fixed hai aur change nahi hona chahiye (immutable)
Set	Sirf unique values chahiye, order matter nahi karta
Dictionary	Key-value pairs — kisi cheez ko naam se fast lookup karna hai
Stack	Last-in-first-out — undo operation, bracket matching, recursion tracking
Queue	First-in-first-out — task scheduling, order processing

Wrong Approach → Better Approach Example

Problem: List mein duplicates hai ya nahi check karna.

Wrong Approach: Nested loop se har element ko har dusre element se compare karna — O(n^2), bade data pe slow.

```
# Wrong (slow) approach
def has_dup_slow(arr):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
```

Better Approach: Set use karo — O(1) lookup, poora check O(n) mein.

```
# Better (fast) approach
def has_dup_fast(arr):
    return len(arr) != len(set(arr))
```

Why? Set mein value dhundna average case mein O(1) hai (hashing ki wajah se), list mein O(n). Isliye set ka use nested loop ko replace kar deta hai.

LEVEL 8

The Universal Problem-Solving Framework

One framework you can apply to any programming problem

Step 1. What is the problem?
Step 2. What are the inputs?
Step 3. What should be the output?
Step 4. What are the constraints?
Step 5. Can I solve it manually?
Step 6. Can I break it into smaller problems?
Step 7. What data structure should I use?
Step 8. What algorithm should I use?
Step 9. Can I write pseudocode?
Step 10. Can I dry run it?
Step 11. Can I code it?
Step 12. Can I optimize it?

Applied Example: "Find the First Non-Repeating Character in a String"

- **Problem:** String mein pehla aisa character dhundo jo sirf ek baar aaya ho.
- **Inputs:** Ek string.
- **Output:** Wo character (ya "none" agar nahi mila).
- **Constraints:** Case-sensitive hoga, empty string bhi ho sakti hai.
- **Manually solve:** Haath se ek example try karo — "swiss" → s(2), w(1), i(1) → 'w' first non-repeating.
- **Break into smaller problems:** (1) Har character ka count nikalo, (2) String ko order mein traverse karke pehla count==1 wala dhundo.
- **Data structure:** Dictionary (character → count).
- **Algorithm:** Ek pass mein counts banao, dusre pass mein first count==1 dhundo.

```
def first_unique_char(s):
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in s:
        if counts[ch] == 1:
            return ch
    return None

print(first_unique_char("swiss"))  # 'w'
```

Optimize: Yeh already O(n) hai — do passes lekin dono linear, isse better complexity possible nahi (poori string toh dekhni hi padegi).

Part 9 — Dry Run Training

Dry run matlab code ko haath se, line-by-line, table bana kar chalana — bina computer ke. Yeh sabse powerful debugging aur understanding skill hai.

Example 1 (Easy):

```
total = 0
for i in range(1, 6):
    total += i
```

i	total before	calculation	total after
1	0	0+1	1
2	1	1+2	3
3	3	3+3	6
4	6	6+4	10
5	10	10+5	15

Example 2 (Medium — while loop with break):

```
n = 29
i = 2
while i * i <= n:
    if n % i == 0:
        print("Not prime")
        break
    i += 1
else:
    print("Prime")
```

i	i*i <= n?	n % i	Action
2	4<=29 Yes	1	Continue, i=3
3	9<=29 Yes	2	Continue, i=4
4	16<=29 Yes	1	Continue, i=5
5	25<=29 Yes	4	Continue, i=6
6	36<=29 No	-	Loop ends → else runs → 'Prime'

Example 3 (Harder — nested loop):

```
result = []
for i in range(1, 4):
    for j in range(1, 3):
        result.append(i * j)
```

i	j	i*j	result after append
1	1	1	[1]

1	2	2	[1, 2]
2	1	2	[1, 2, 2]
2	2	4	[1, 2, 2, 4]
3	1	3	[1, 2, 2, 4, 3]
3	2	6	[1, 2, 2, 4, 3, 6]

Part 10 — Debugging & Logic Errors

Error Type	Kya Hota Hai	Example
Syntax Error	Python ke grammar rules break hote hain	missing colon: if x > 5
Runtime Error	Code chalte waqt crash hota hai	10 / 0 (ZeroDivisionError)
Logical Error	Code chalta hai lekin galat answer deta hai	average = sum + count (÷ ki jagah +)
Infinite Loop	Loop kabhi nahi rukta	counter kabhi update nahi hota
Wrong Condition	Galat comparison operator	> ki jagah >= chahiye tha
Off-by-one	Loop ek baar zyada/kam chalta hai	range(1, n) instead of range(1, n+1)

Buggy Code — Bug Dhundo (Answer neeche hai, pehle khud try karo)

```
# Goal: print 1 to 5
i = 1
while i < 5:
    print(i)
    i += 1
```

Hint: Loop kitni baar chalega, dry run karke dekho.

Bug: Off-by-one error — i < 5 ki wajah se 5 print nahi hota. Fix: while i <= 5:

Part 11 — Final Projects (Beginner → Advanced)

Beginner

- Calculator
- Number Guessing Game
- Grade Calculator
- Simple ATM

Intermediate

- Expense Tracker
- Contact Book
- Quiz App
- Student Management System

Advanced

- CLI Banking System
- Inventory Management System
- Expense Tracker with File Storage
- Mini API-oriented Backend Project

Part 12 — 15-Day Logic-Building Roadmap

Day	Focus Area
Day 1	Programming logic ka intro + 14-step problem solving process
Day 2	Conditions — if/elif/else, comparison problems (Level 1)
Day 3	Loops — for/while, counters, accumulators (Level 2)
Day 4	Number problems — factorial, prime, palindrome, Fibonacci
Day 5	Pattern problems — star & number patterns
Day 6	Strings — traversal, frequency counting, palindrome, anagram
Day 7	Lists & Tuples — searching, sorting logic, second largest
Day 8	Dictionaries & Sets — frequency maps, deduplication
Day 9	Functions — modular thinking, Expense Tracker project
Day 10	Intermediate project #1 (Contact Book or Quiz App)
Day 11	Intermediate project #2 (Student Management System)
Day 12	Debugging practice — syntax, runtime, logical errors
Day 13	Searching & Sorting algorithms + Big O basics

Day 14	Recursion + Two Pointers + Sliding Window + Hashing
Day 15	Universal Framework applied to 3 mixed-difficulty problems + review

Part 13 — Final Problem-Solving Checklist

Kisi bhi naye problem ko solve karne se pehle yeh checklist use karo:

- ■ Kya maine problem 2 baar padha hai?
- ■ Kya main problem apne shabdon mein explain kar sakta hoon?
- ■ Kya mujhe pata hai input kya hai aur uska type kya hai?
- ■ Kya mujhe pata hai exact output kya chahiye?
- ■ Kya maine saari conditions/edge cases list ki hain (empty, 0, negative, duplicate)?
- ■ Kya maine problem ko chhote parts mein toda hai?
- ■ Kya maine zaroori variables identify kiye hain?
- ■ Kya maine ek plain-English algorithm likha hai?
- ■ Kya maine pseudocode likha hai code se pehle?
- ■ Kya maine kam se kam ek example haath se dry run kiya hai?
- ■ Kya code likhne ke baad maine normal cases test kiye?
- ■ Kya maine edge cases bhi test kiye?
- ■ Kya is solution ko simpler ya faster banaya ja sakta hai?
- ■ Kya maine socha hai ki iske liye sahi data structure kaunsa hai?

Yeh guide ek foundation hai. Asli practice interactive session mein hogi — jahan har problem par pehle tumhe sochne ka mauka milega, mistakes par hints milenge, aur last mein sahi logic aur code dikhaya jayega. Happy problem solving!